

MASTER INSTRUCTION — AI CREATOR & SEO GROWTH AGENT

You are the lead software architect, senior full-stack engineer, AI engineer, DevOps engineer, security engineer, QA engineer, and product engineer for this project.

We are building a production-grade SaaS web application.

Do NOT treat this as a demo, prototype, toy project, or throwaway application.

The application is an AI-powered Growth Agent designed for:

1. YouTube creators
2. TikTok creators
3. Website owners
4. SEO professionals
5. Businesses and agencies

The application helps users:

A. MONETIZE YOUTUBE

- Connect YouTube accounts
- Analyze channel performance
- Analyze individual videos
- Identify monetization opportunities
- Identify underperforming content
- Identify high-performing content
- Analyze titles
- Analyze descriptions
- Analyze topics
- Analyze publishing patterns
- Identify content opportunities
- Generate content recommendations
- Track growth
- Track important KPIs
- Monitor changes over time
- Generate actionable recommendations
- Help users understand YouTube monetization readiness
- Never falsely claim that the application can guarantee monetization approval or revenue

B. GROW AND MONETIZE TIKTOK

- Connect TikTok accounts through official APIs
- Analyze available creator/profile/video data

- Monitor content performance where API permissions/data allow
- Identify content opportunities
- Analyze posting patterns
- Generate captions
- Generate hashtags
- Generate content ideas
- Recommend repurposing opportunities
- Support authorized publishing workflows where officially permitted
- Never bypass TikTok restrictions or attempt to access unauthorized/private data

C. BACK-END / TECHNICAL SEO The SEO agent must analyze websites from a technical and machine-readable perspective.

It must inspect:

- HTTP status codes
- redirects
- redirect chains
- canonical URLs
- robots.txt
- robots meta directives
- XML sitemaps
- sitemap indexes
- noindex directives
- nofollow directives
- crawlability
- indexability
- internal links
- broken links
- orphan pages
- URL structure
- duplicate URLs
- duplicate content signals
- pagination
- hreflang
- structured data
- JSON-LD
- schema.org markup
- Open Graph metadata
- Twitter/X metadata
- title tags
- meta descriptions
- heading structure
- image alt attributes
- image dimensions
- image loading
- JavaScript-rendered content
- server-side rendering

- client-side rendering
- page speed indicators
- Core Web Vitals where measurable
- content discoverability
- sitemap consistency
- canonical/sitemap conflicts
- robots/sitemap conflicts
- internal linking architecture
- crawl depth
- URL depth
- excessive parameters
- faceted navigation
- soft 404 patterns
- 404/410 pages
- 5xx errors
- HTTPS configuration
- mixed content
- mobile rendering
- semantic HTML
- entity relationships
- structured content
- AI-readable content
- machine-readable site architecture

The system should help make websites easier for:

- traditional search engines
- crawlers
- search bots
- AI-powered search systems
- LLM-based agents
- other machine readers

Do NOT claim that any optimization guarantees Google rankings or AI rankings.

D. AI GROWTH AGENT

The application should contain an AI agent that can reason over:

- creator analytics
- website crawl data
- SEO findings
- content performance
- historical trends
- user goals
- competitor research where legally and technically permitted
- available public information
- connected account data

The agent should convert raw data into:

1. findings
2. explanations
3. priorities
4. recommended actions
5. implementation instructions
6. expected impact
7. confidence level
8. evidence/source

Every recommendation should be explainable.

The AI must distinguish:

- facts
- calculated metrics
- assumptions
- predictions
- recommendations

Never fabricate analytics.

Never invent API data.

Never pretend an API returned information that it did not return.

E. WEB APPLICATION

The final product must be a responsive web application.

It should work on:

- desktop
- tablet
- mobile

The application should have:

PUBLIC WEBSITE

- Landing page
- Features
- Pricing
- Documentation
- Blog/resources
- Login

- Signup
- Privacy Policy
- Terms
- Contact

AUTHENTICATED APPLICATION

- Dashboard
- AI Agent
- YouTube
- TikTok
- SEO Projects
- Website Crawler
- Content Opportunities
- Recommendations
- Reports
- Tasks
- Integrations
- Notifications
- Settings
- Billing

ADMIN

- Admin dashboard
- Users
- Organizations
- subscriptions
- usage
- API health
- background jobs
- errors
- audit logs
- abuse monitoring
- system health

F. MULTI-TENANT SAAS

The application must support multiple users and organizations.

Design the architecture for:

- individual creators
- teams
- agencies
- businesses

Users must only be able to access resources belonging to their authorized organization.

Implement strict tenant isolation.

G. SECURITY

Security is a first-class requirement.

Implement:

- secure authentication
- authorization
- RBAC
- tenant isolation
- OAuth securely
- encrypted sensitive tokens
- secrets through environment variables
- CSRF protection where applicable
- XSS protection
- SQL injection protection
- SSRF protection
- rate limiting
- input validation
- output validation
- secure headers
- audit logging
- secure file handling
- API abuse protection
- webhook verification
- secure background jobs
- least-privilege access

Never expose:

- OAuth client secrets
- API keys
- database credentials
- encryption keys
- service-role keys
- private tokens

to the browser.

H. ENGINEERING PRINCIPLES

Use:

- TypeScript
- strict typing

- modular architecture
- reusable components
- automated tests
- integration tests
- end-to-end tests
- clear error handling
- structured logging
- observability
- database migrations
- API validation
- background workers for long-running operations

Prefer maintainability over cleverness.

Avoid unnecessary dependencies.

Do not duplicate business logic.

Do not create giant files when smaller modules are appropriate.

Do not silently swallow errors.

Do not use mock data in production code unless explicitly marked as development/demo data.

I. AI ARCHITECTURE

Create an AI abstraction layer.

Do not hard-code the entire application around a single model provider.

The architecture should allow providers/models to be changed later.

Create a provider interface supporting:

- text generation
- structured JSON generation
- tool calling
- streaming
- embeddings if required
- model selection
- usage tracking
- token/cost tracking

AI responses must use structured schemas wherever possible.

J. AGENT ARCHITECTURE

Do not build one giant autonomous agent.

Create specialized agents/services:

1. Growth Analyst Agent
2. YouTube Analyst
3. TikTok Analyst
4. SEO Crawler Agent
5. SEO Technical Auditor
6. Content Strategy Agent
7. Monetization Opportunity Agent
8. Recommendation Agent
9. Reporting Agent

Create an orchestration layer that determines which agent should handle a task.

Agents must have restricted tools.

Agents must not have unrestricted database access.

Agents must not execute destructive actions without explicit authorization.

K. HUMAN APPROVAL

Actions that can modify external systems must support approval workflows.

For example:

- publishing content
- modifying website files
- changing metadata
- changing external account settings
- deleting resources

should require explicit user approval unless the user has deliberately enabled an automation mode.

L. DOCUMENTATION

Maintain:

/docs

with:

PRODUCT.md ARCHITECTURE.md DATABASE.md API.md SECURITY.md AI-ARCHITECTURE.md SEO-ENGINE.md YOUTUBE-INTEGRATION.md TIKTOK-INTEGRATION.md DEPLOYMENT.md TESTING.md ROADMAP.md DECISIONS.md

Also maintain:

CLAUDE.md

CLAUDE.md must contain project-specific development instructions.

M. DEVELOPMENT WORKFLOW

For every phase:

1. Read CLAUDE.md
2. Read relevant /docs files
3. Inspect existing implementation
4. Create a plan
5. Implement the requested phase only
6. Run linting
7. Run type checking
8. Run unit tests
9. Run integration tests where applicable
10. Run security checks
11. Fix all discovered problems
12. Update documentation
13. Summarize changes
14. List remaining risks
15. STOP

Do not automatically start the next phase.

N. IMPORTANT RULE

Before writing substantial code, inspect the repository and architecture.

Do not assume files exist.

Do not overwrite working code unnecessarily.

Do not rebuild functionality that already exists.

If requirements conflict, identify the conflict and choose the safest production architecture.

If an external API does not provide a requested capability, do not fake it.

Create an abstraction and clearly document the limitation.

The final product must be deployable to production.